

CareLens AI

Shared team specification — 8 October 2026

Evidence-backed patient history: Compare → Verify → Update.

Team: Varun Yadav T — backend; Sakthivel — AI/RAG; Rakshana — frontend.

This document supersedes earlier planning documents for stack and ownership. It specifies the intended hackathon build. The starter package currently implements only the FastAPI health endpoint, a frontend connection screen, and initial Q&A schemas. Other features below are not completed.

1. Idea and purpose

CareLens helps authorized clinic staff reconstruct a patient's history from scattered consultation notes, reports, medication records, procedures, and follow-ups.

The staff member selects a patient, views a timeline, asks a question, and verifies the answer against the original PDF page. The signature demonstration compares two recorded treatment cycles, identifies an unmatched test request, then ingests a new report and shows the supported change.

Use synthetic data only. The system organizes and retrieves recorded information; clinicians make medical decisions. Novelty and impact must be demonstrated rather than claimed.

2. Final stack

Layer	Selected technology	Purpose
Frontend	React + Vite + TypeScript	Patient workspace
UI	Tailwind CSS + shadcn/ui	Consistent components
Backend	Python + FastAPI + Pydantic	APIs, validation and services
Configuration	pydantic-settings	Environment configuration
Database	Supabase PostgreSQL	Patients, grants, facts and audit events
Vector search	pgvector in the same PostgreSQL database	Patient-filtered semantic search
Database access	Supabase Python client and SQL RPC functions	CRUD and transactional ingestion/search operations
Login	Supabase Auth	Staff sessions and access tokens
Permissions	Backend roles + per-patient action grants; database RLS	Authorization
Original documents	Supabase private Storage	Immutable synthetic PDFs
PDF extraction	pdfplumber	Page-preserving text extraction
Evidence display	PDF.js	Original page viewer
Embeddings	Sentence Transformers locally	Document/query vectors
Primary LLM	Gemini API via google-genai	Extraction, summaries and cited answers
Fallback LLM	Groq API via groq SDK	Provider-outage fallback
Verification	pytest + HTTPX; Playwright	Logic/API/access and browser flows
Packaging	Docker	Reproducible backend
Version control	Git + GitHub	Shared source and reviews
Initial execution	Local frontend/API + hosted Supabase	Development and laptop demonstration

Embedding candidate: sentence-transformers/all-MiniLM-L6-v2 (384 dimensions). Accept it only after testing retrieval on our dummy clinical records. Use the same model/version for document and query embeddings. Changing it requires re-embedding.

Provider choices are settled; exact Gemini and Groq model IDs must be configured after checking account availability, quota, structured-output support, and a small evaluation. Do not invent model names or

assume every model supports the same schema features.

Hosting: build and test locally first. Vercel Hobby is a candidate for an eligible noncommercial frontend; public backend hosting remains undecided until deployment requirements are confirmed. A hosted frontend cannot use localhost to reach the team's laptop for other visitors.

Free-tier-first is a budget objective, not a promise of unlimited usage. Check all account quotas before the demo. No paid service is required by this specification.

3. Scope

P0: first working milestone

- Supabase staff login.
- Patient directory filtered by permissions.
- Upload a text-based PDF, extract text and preserve pages.
- Persist chunks and vectors.
- Patient timeline and evidence-backed case brief.
- Patient-scoped Q&A with claim-linked citations.
- Authorized original-PDF viewing.
- Basic access audit events, loading/error/denied states.

P1: signature demonstration

- Compare two explicitly recorded cycles.
- Match test requests to results conservatively.
- Upload a new report, reconcile it, increment record version, and display changes.

P2: only after P0/P1 work

- Narrow conflict detection/review, starting with different dates for the same identifiable procedure.
- Date/type filters and improved source highlighting.

Defer OCR unless actual supplied records are scanned. Defer voice, multilingual queries, patient chatbot, advanced dashboards and custom model training.

No diagnosis, treatment recommendations, invented missing facts or silent conflict resolution.

4. Team ownership

Member	Owns	First output
Varun	API routes, identity verification, patient grants, DB/RPCs, private uploads, matching, version/change persistence and audits	Verified login → permitted patient list
Sakthivel	PDF parser, chunking, fact extraction, embeddings, retrieval, Gemini/Groq adapter, cited generation and cycle-comparison logic	Document text → retrievable evidence → cited answer
Rakshana	React login, directory, workspace, timeline, comparison, evidence drawer, upload/change states and API integration	Real frontend → backend connection, then login/directory
All	Contracts, fixtures, tests, integration, pitch and demo	One stable end-to-end flow

Varun authorizes requests before invoking Sakthivel's services. Sakthivel accepts an authorized patient scope, not an unrestricted record collection. Rakshana consumes agreed contracts; UI visibility does not replace backend enforcement.

Varun and Sakthivel agree the fact schema, repository interface and ingestion commit boundary before coding independently. Varun implements matching/update persistence; Sakthivel supplies extracted facts and provenance.

5. Architecture

```
flowchart TD
    UI["React workspace"] --> AUTH["Supabase Auth: login"]
    UI --> API["FastAPI: verified identity and patient grants"]
    API --> ING["Ingestion: pages, chunks and facts"]
    API --> RET["Patient-filtered retrieval and comparison"]
    API --> REC["Matching, changes and audits"]
    ING --> STORE["Private PDF storage"]
    ING --> DB["PostgreSQL and pgvector"]
    RET --> DB
    REC --> DB
    ING --> AI["Gemini primary / Groq fallback"]
    RET --> AI
    API --> STORE
```

React uses Supabase only for identity/session handling. Business data, document viewing and LLM calls go through FastAPI. Local embeddings run in the backend ingestion/retrieval modules; they do not depend on the answer provider.

Ingestion

Authorize upload → validate file → hash/deduplicate → store original → extract per page → chunk → extract candidate facts with evidence → validate → embed → transactionally persist → reconcile → publish new record version/change record.

Exclude incomplete ingestion from retrieval. Failures are visible and retryable. Initial in-process processing is acceptable for small fixtures, but is not a durable queue; on restart, stale jobs must become retryable.

Question answering

Verify token → check patient/action grant → structured and keyword/vector retrieval inside that patient scope → build bounded evidence context → primary LLM → validate schema and sources → return claims/citations/limitations → record audit event.

Missing evidence produces `insufficient_evidence`. Valid source IDs alone do not prove claim support; evaluate claims against original excerpts.

6. Two-provider policy

- Primary: Gemini. Fallback: Groq.
- A single provider adapter returns the same validated schemas.
- On primary timeout, rate limit, connection failure, or retryable server error, try fallback once.
- Suggested initial budget: 15 seconds per attempt, 35 seconds overall including overhead; tune using measured extraction and Q&A latency.
- Do not fallback for insufficient evidence, permission denials, safety refusal or an incorrect API key. Surface configuration errors clearly.
- Invalid output must never be displayed as a valid answer. Return a controlled validation failure; any bounded repair policy requires separate tests.
- Use exactly the same authorized evidence and record version for both providers.
- If both fail, display service unavailable; existing structured records remain usable.
- Persist provider/model metadata for evaluation. Do not log secrets or unnecessary full prompts.
- No second-provider call occurs unless fallback is needed.
- Switching LLM provider does not change local embeddings or the vector index.

Implement the primary path first, then test fallback by deliberately simulating provider failure.

7. Identity and permissions

Initial application roles: doctor, coordinator, admin. Lab/reviewer roles are backlog items.

Role	Allowed actions, subject to patient grants
Doctor	Read, Q&A, summary, comparison, review
Coordinator	Read, upload, view reconciliation and follow-ups
Admin	Manage staff/grants, patient metadata and audits; clinical read/upload/review require grants

Seed staff and grants initially; an admin dashboard is unnecessary. No unrestricted self-registration or client-controlled role assignment.

Supabase Auth owns passwords and token issuance. FastAPI uses supported verified-token handling and validates the expected project/issuer, audience and expiry. Never trust unverified JWT decoding or frontend role fields.

Authorize every document, status, comparison and change endpoint. Filter by patient before evidence reaches either LLM. RLS adds protection when requests execute as the user. Service-role access bypasses RLS, so privileged backend paths must explicitly enforce permissions; carefully scope SQL RPCs and validate their caller/context. Use short-lived document links after authorization.

Treat uploaded text as untrusted evidence, never executable instructions. Permission boundaries are enforced in code, not by prompts.

8. Record semantics

- `recorded_pending`: a source explicitly states pending.
- `result_not_found`: a request exists, but no corresponding result was found in available records.
- `matched`: evidence supports a specific request/result link.
- `needs_review`: ambiguous matches or contradictory facts.

Absence of a result does not establish that the test was never performed.

Match patient identity first; prefer explicit request/test identifiers. Use test type, recorded cycle and dates as supporting criteria. Repeated tests must not match by name alone. Conflicting patient identifiers require rejection or review.

Cycles must come from recorded identifiers or staff review. Different dates are conflicts only when they refer to the same identifiable event.

9. Minimum entities

Table	Essential fields
<code>staff_profiles</code>	<code>auth_user_id</code> , <code>role</code>
<code>patients</code>	<code>id</code> , <code>synthetic_name</code> , <code>record_version</code>
<code>patient_grants</code>	<code>user_id</code> , <code>patient_id</code> , <code>allowed_actions</code>
<code>documents</code>	<code>id</code> , <code>patient_id</code> , <code>storage_key</code> , <code>hash</code> , <code>status</code> , <code>uploader</code> , <code>timestamp</code>
<code>chunks</code>	<code>id</code> , <code>patient_id</code> , <code>document_id</code> , <code>page</code> , <code>content</code> , <code>embedding</code> , <code>model_version</code>
<code>events</code>	<code>id</code> , <code>patient_id</code> , <code>type</code> , <code>date nullable</code> , <code>cycle_id nullable</code> , <code>payload</code>
<code>fact_sources</code>	<code>event/fact id</code> , <code>document id</code> , <code>page</code> , <code>chunk id</code> , <code>excerpt</code>
<code>test_requests</code>	<code>id</code> , <code>patient_id</code> , <code>request_identifier</code> , <code>test_type</code> , <code>date</code> , <code>cycle_id</code> , <code>source</code>
<code>test_results</code>	<code>id</code> , <code>patient_id</code> , <code>request_identifier</code> , <code>test_type</code> , <code>date</code> , <code>cycle_id</code> , <code>source</code>
<code>request_result_links</code>	<code>request_id</code> , <code>result_id</code> , <code>state</code> , <code>match_basis</code> , <code>evidence</code>

Table	Essential fields
conflicts	patient_id, fact references, kind, review_state
record_changes	patient_id, from_version, to_version, document_id, supported changes
audit_events	actor, action, patient/resource, time, outcome

Publish versions atomically after successful ingestion. Mark cached answers/briefs stale after a version change. Human review creates a separate interpretation; it does not destroy the original assertions.

10. API contract

Supabase SDK handles login; do not implement a second JWT issuer.

Method	Route
GET	/health
GET	/api/v1/patients
GET	/api/v1/patients/{id}
POST	/api/v1/patients/{id}/documents
GET	/api/v1/patients/{id}/documents
GET	/api/v1/patients/{id}/documents/{doc_id}/status
GET	/api/v1/patients/{id}/documents/{doc_id}/view
GET	/api/v1/patients/{id}/timeline
GET	/api/v1/patients/{id}/brief
POST	/api/v1/patients/{id}/questions
GET	/api/v1/patients/{id}/cycles/compare?cycle_a=C1&cycle_b=C2
GET	/api/v1/patients/{id}/reconciliation
GET	/api/v1/patients/{id}/changes?since_version=1
POST	/api/v1/patients/{id}/conflicts/{conflict_id}/reviews

Every business route is protected. Use consistent UUID identifiers internally; display P001-style labels separately. Distinguish 401 unauthenticated, 403 forbidden, 422 invalid input, and 503 unavailable.

Question body:

```
{"question":"Compare the last two recorded cycles."}
```

Response contract:

```
{
  "patient_id":"<patient UUID>",
  "record_version":2,
  "status":"answered",
  "answer":"Test A was requested in cycle C2.",
  "claims":[{"text":"Test A was requested in cycle C2.", "source_ids":["S1"]}],
  "sources":[{"source_id":"S1",
    "document_id":"<document UUID>",
    "document_name":"cycle_2_note.pdf",
    "page":1,
    "excerpt":"Cycle C2: Test A requested."
  }],
  "limitations":[]
}
```

Statuses: answered, partial, insufficient_evidence. Each factual claim needs supporting sources. All source IDs map to authorized retrieved evidence. No arbitrary confidence percentage.

11. Environment templates

Backend:

```
CORS_ORIGINS=["http://localhost:5173"]
SUPABASE_URL=
SUPABASE_PUBLISHABLE_KEY=
SUPABASE_SERVICE_ROLE_KEY=
```

```

STORAGE_BUCKET=patient-documents
PRIMARY_LLM_PROVIDER=gemini
GEMINI_API_KEY=
GEMINI_MODEL=
FALLBACK_LLM_PROVIDER=groq
GROQ_API_KEY=
GROQ_MODEL=
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
LLM_TIMEOUT_SECONDS=15
LLM_TOTAL_TIMEOUT_SECONDS=35
MAX_UPLOAD_BYTES=10485760
MAX_QUESTION_CHARS=2000

```

Frontend:

```

VITE_API_BASE_URL=http://localhost:8000
VITE_SUPABASE_URL=
VITE_SUPABASE_PUBLISHABLE_KEY=

```

These names are the agreed target configuration; implement settings fields before expecting them to take effect. Backend secrets never use VITE_ prefixes. Commit placeholders only. Lock tested dependencies after installation.

12. Repository boundaries

- frontend/src/: Rakshana; pages, workspace components, API client and auth.
- backend/app/api/, auth/, database/, reconciliation/, audit/: Varun.
- backend/app/ingestion/, retrieval/, providers/: Sakthivel.
- backend/app/schemas/: shared contract, Varun integrates.
- database/migrations/: Varun; transactional functions and indexes.
- demo-data/: shared synthetic fixtures and expected answers.
- docs/: shared architecture, evaluation and pitch.
- backend/tests/: unit, integration and access checks.
- frontend/e2e/: browser workflow checks.

Avoid duplicate definitions across modules. Use feature/backend-security, feature/ai-rag and feature/frontend branches; review and merge small changes often.

13. Run the existing starter on Windows

Extract CareLens_AI_Starter.zip and open the carelens-ai folder in Antigravity. Python 3.11+ and a Vite-compatible Node version are required; Node 22.12+ is a suitable baseline.

Backend terminal:

```

cd backend
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
Copy-Item .env.example .env
.\.venv\Scripts\python.exe -m uvicorn app.main:app --reload --port 8000

```

Frontend terminal, from repository root:

```

cd frontend
Copy-Item .env.example .env
npm ci
npm run dev

```

Open <http://localhost:5173> and select Check backend. Starter includes a package lock, so npm ci is reproducible. Frontend build and backend health/fail-closed checks were verified; a full browser end-to-end test was not performed.

Starter requirements intentionally omit future service dependencies. Add google-genai, groq, sentence-transformers, pdfplumber, supabase and test packages in the relevant work branches; install and lock versions after verification. This README does not imply those integrations exist.

14. Build checkpoints

1. Setup: both apps run, backend connection succeeds.
2. Access: real Supabase session, verified token, permitted patient list, denied unauthorized access.
3. Evidence: actual PDF ingestion, stored chunks, question, supported answer, original source page.
4. Differentiation: timeline/cycle comparison, conservative test matching, new-report update.
5. Validation: second patient, paraphrased questions, uncertain evidence, repeat tests and provider outage.
6. Demo: measured impact, slides, recorded backup and actual laptop rehearsal.

Schedule: finish the working demo on 9 October evening. First evaluation starts 10 October at 9 AM; use the 11 AM–1 PM implementation window for feedback/fixes. Submission deadline and judging rubric remain unconfirmed.

15. Acceptance and demo

Seed three synthetic patients and two accounts with different grants. Main case: two labeled cycles, repeated test requests with distinct IDs, one unmatched request, and an optional identifiable date conflict. Keep the later result PDF outside initial ingestion.

Demo: login → patient → question → source page → cycle comparison → unmatched request → upload report → updated match/change summary → unauthorized access denied.

Check:

- Invalid/expired tokens rejected.
- Other patients never enter retrieval context.
- Direct PDF/status endpoints enforce permissions.
- Unsupported questions remain uncertain.
- Every citation resolves to correct evidence.
- Repeated tests and ambiguous matches handled correctly.
- Duplicate files do not duplicate facts.
- Failed uploads do not corrupt prior history.
- Record changes invalidate stale output.
- Gemini outage triggers Groq; missing evidence does not.
- Both-provider failure yields an honest error.
- Another patient and a paraphrase work without hardcoded answers.

Prepare an answer key before measuring support/citation accuracy. Measure manual versus assisted lookup on equivalent tasks; report sample size, conditions, limitations and failures. Separate ingestion time from answer latency. Do not invent accuracy or time-saving claims.

16. Official technical references

- Gemini structured outputs (<https://ai.google.dev/gemini-api/docs/structured-output>)
- Gemini API pricing (<https://ai.google.dev/gemini-api/docs/pricing>)
- Groq structured outputs (<https://console.groq.com/docs/structured-outputs>)
- Groq rate limits (<https://console.groq.com/docs/rate-limits>)
- Supabase verified JWT claims (<https://supabase.com/docs/reference/python/auth-getclaims>)
- Supabase plans (<https://supabase.com/pricing>)
- Vite getting started (<https://vite.dev/guide/>)
- FastAPI CORS (<https://fastapi.tiangolo.com/tutorial/cors/>)

Pitch

“CareLens turns scattered patient records into a verifiable case history, showing what happened, what remains unresolved, and what changed when new evidence arrived.”